

Feedback

Assessed on 13. april 2026 09:21 by  Yoganata Kristanto

Your answer COMPLETED

Submitted: 18. marts 2026 15:20 by  Toke Odgaard-Jans

Statemachonk

```
/*
```

```
* generated by Xtext 2.42.0
```

```
*/
```

```
package org.xtext.example.statedsl.generator;
```

```
import org.eclipse.emf.ecore.resource.Resource;
```

```
import org.eclipse.xtext.generator.AbstractGenerator;
```

```
import org.eclipse.xtext.generator.IFileSystemAccess2;
```

```
import org.eclipse.xtext.generator.IGeneratorContext;
```

```
import org.xtext.example.statedsl.stateDSL.AState;
```

```
import org.eclipse.xtext.naming.IQualifiedNameProvider;
```

```
import jakarta.inject.Inject;
```

```
import org.xtext.example.statedsl.stateDSL.State;

import org.xtext.example.statedsl.stateDSL.Transition;

import org.xtext.example.statedsl.stateDSL.StateMachine;

import org.xtext.example.statedsl.stateDSL.Condition;


/**
 * Generates GDScript code from your model files on save.
 */

class StateDSLGenerator extends AbstractGenerator {

@Inject

extension IQualifiedNameProvider;

override void doGenerate(Resource resource, IFileSystemAccess2 fsa, IGeneratorContext context) {

val stateMachine = resource.allContents.tolterable.filter(StateMachine).head; // Get the first StateMach
```

```
if (stateMachine === null) {

System.err.println("Error: No StateMachine found in the resource.");

return; // Exit if no state machine exists

}

if (stateMachine.states.isEmpty()) {

System.err.println("Error: State machine '" + stateMachine.name + "' must have at least one state.");

return; // Exit if the state machine has no states

}

fsa.generateFile(

stateMachine.fullyQualifiedName.toString("/") + ".gd",

compileBase(stateMachine)

);

}

private def compileBase(StateMachine machine) ""
```

```
extends Node2D
```

```
class_name Base
```

```
var baseMachine: StateMachine = StateMachine.new()
```

```
@onready var base = $"."
```

```
func _ready() -> void:
```

```
add_child(baseMachine)
```

```
baseMachine\
```

```
«FOR state : machine.states»
```

```
«IF state instanceof State»
```

```
.AddState("«state.name»", «state.name».new())\
```

```
«ELSEIF state instanceof StateMachine»
```

```
«(state as StateMachine).compile»
```

```
«ENDIF»
```

```
«ENDFOR»\

«FOR transition : machine.transitions»

.AddTransition("«transition.from.name»", "«transition.to.name»", Callable(self, "«transition.condition.name»"))

«ENDFOR»\

.SetStart("«machine.start.name»")\

.Build()

baseMachine.Entry()

'''

private def compile(StateMachine machine) '''

.AddState("«machine.name»", StateMachine.new()\

«FOR state : machine.states»

«IF state instanceof State»

.AddState("«state.name»", «state.name».new())\

«ELSEIF state instanceof StateMachine»

«(state as StateMachine).compile»
```

```
«ENDIF»
```

```
«ENDFOR»\
```

```
«FOR transition : machine.transitions»
```

```
.AddTransition("«transition.from.name»", "«transition.to.name»", Callable(self, "«transition.condition.name»"))
```

```
«ENDFOR»\
```

```
.SetStart("«machine.start.name»")\
```

```
""
```

```
}
```

code generation output.pdf

...